



AI-Assisted Verification with Interactive Synthesis

Rafe Symonds

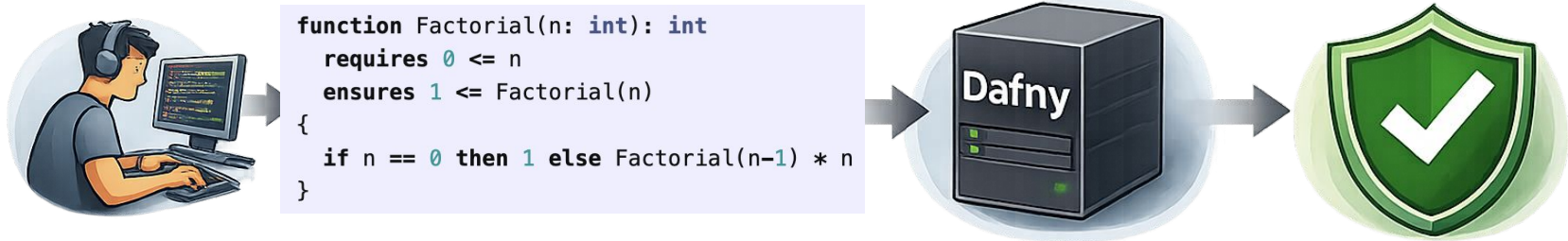
In Dafny writing the code isn't the hard part, it's figuring out the invariants.

```
{
  var t := root;
  var p: Node := null; // parent of t in original graph
  ghost var path := #Path.Empty;
  t.marked := true;
  t.pathFromRoot := path;
  ghost var stackNodes := [];
  ghost var unmarkedNodes := S - {t};
  while (true)
    invariant root.marked ∧ t ≠ null ∧ t ∈ S ∧ t ∉ stackNodes;
    invariant |stackNodes| = 0 ⇔ p = null;
    invariant 0 < |stackNodes| ⇒ p = stackNodes[|stackNodes|-1];
    // stackNodes has no duplicates:
    invariant (∀ i, j • 0 ≤ i ∧ i < j ∧ j < |stackNodes| ⇒ stackNodes[i] ≠ stackNodes[j]);
    invariant (∀ n • n ∈ stackNodes ⇒ n ∈ S);
    invariant (∀ n • n ∈ stackNodes ∨ n = t ⇒
      n.marked ∧ 0 ≤ n.childrenVisited ∧ n.childrenVisited ≤ |n.children| ∧
      (∀ j • 0 ≤ j ∧ j < n.childrenVisited ⇒ n.children[j] = null ∨ n.children[j].marked));
    invariant (∀ n • n ∈ stackNodes ⇒ n.childrenVisited < |n.children|);
    invariant (∀ n • n ∈ S ∧ n.marked ∧ n ∉ stackNodes ∧ n ≠ t ⇒
      (∀ ch • ch ∈ n.children ∧ ch ≠ null ⇒ ch.marked));
    invariant (∀ n • n ∈ S ∧ n ∉ stackNodes ∧ n ≠ t ⇒
      n.childrenVisited = old(n.childrenVisited));
    invariant (∀ n • n ∈ S ⇒ n ∈ stackNodes ∨ n.children = old(n.children));
    invariant (∀ n • n ∈ stackNodes ⇒
      |n.children| = old(|n.children|) ∧
      (∀ j • 0 ≤ j ∧ j < |n.children| ⇒ j = n.childrenVisited ∨ n.children[j] = old(n.children[j])));
    // every marked node is reachable:
    invariant old(ReachableVia(root, path, t, S));
    invariant (∀ n, pth • n ∈ S ∧ n.marked ∧ pth = n.pathFromRoot ⇒ old(ReachableVia(root, pth, n, S)));
    invariant (∀ n • n ∈ S ∧ n.marked ⇒ old(Reachable(root, n, S)));
    // the current values of m.children[m.childrenVisited] for m's on the stack:
    invariant 0 < |stackNodes| ⇒ stackNodes[0].children[stackNodes[0].childrenVisited] = null;
    invariant (∀ k • 0 < k ∧ k < |stackNodes| ⇒
      stackNodes[k].children[stackNodes[k].childrenVisited] = stackNodes[k-1]);
    // the original values of m.children[m.childrenVisited] for m's on the stack:
    invariant (∀ k • 0 ≤ k ∧ k+1 < |stackNodes| ⇒
      old(stackNodes[k].children[stackNodes[k].childrenVisited]) = stackNodes[k+1]);
    invariant 0 < |stackNodes| ⇒
      old(stackNodes[|stackNodes|-1].children[stackNodes[|stackNodes|-1].childrenVisited]) = t;
    invariant (∀ n • n ∈ S ∧ ¬n.marked ⇒ n ∈ unmarkedNodes);
    decreases unmarkedNodes, stackNodes, |t.children| - t.childrenVisited;

    if (t.childrenVisited = |t.children|) {
      // pop
      t.childrenVisited := 0;

      if (p = null) {
        return;
      }
      var oldP := p.children[p.childrenVisited];
      p.children := p.children[..p.childrenVisited] + [t] + p.children[p.childrenVisited + 1..];
      t := p;
      p := oldP;
      stackNodes := stackNodes[..|stackNodes| - 1];
      t.childrenVisited := t.childrenVisited + 1;
      path := t.pathFromRoot;
    } else if (t.children[t.childrenVisited] = null ∨ t.children[t.childrenVisited].marked) {
      // just advance to next child
      t.childrenVisited := t.childrenVisited + 1;
    } else {
      // push
      var newT := t.children[t.childrenVisited];
      t.children := t.children[..t.childrenVisited] + [p] + t.children[t.childrenVisited + 1..];
      p := t;
      stackNodes := stackNodes + [t];
      path := #Path.Extend(path, t);
      t := newT;
      t.marked := true;
      t.pathFromRoot := path;
      unmarkedNodes := unmarkedNodes - {t};
    }
  }
}
```

Verification is largely automated, but it depends on human-supplied invariants.



Roadmap

- Foundations
- Automated Checking
- Machine Learning Synthesis
 - Pre-LLM Synthesis
 - LLM Synthesis
- Interactive Assistance



Foundations and Automated Checking



Hoare (1969) established the mathematical foundation for program correctness, but intermediate assertions still require human insight

$$P \{Q\} R$$

P: Precondition

Q: Program

R: Result of the program's execution

If $\vdash P\{Q_1\}R_1$ and $\vdash R_1\{Q_2\}R$ then $\vdash P\{(Q_1 ; Q_2)\}R$

Dafny (2010) automates proof checking via SMT solvers, but it requires human effort to write invariants

```
method ComputeSum(n: nat) returns (sum: nat)
  ensures sum == n
{
  var i := 0;
  sum := 0;
  while i < n {
    i := i + 1;
    sum := sum + 1;
  }
  // Dafny fails to prove the postcondition (sum == n)
}
```

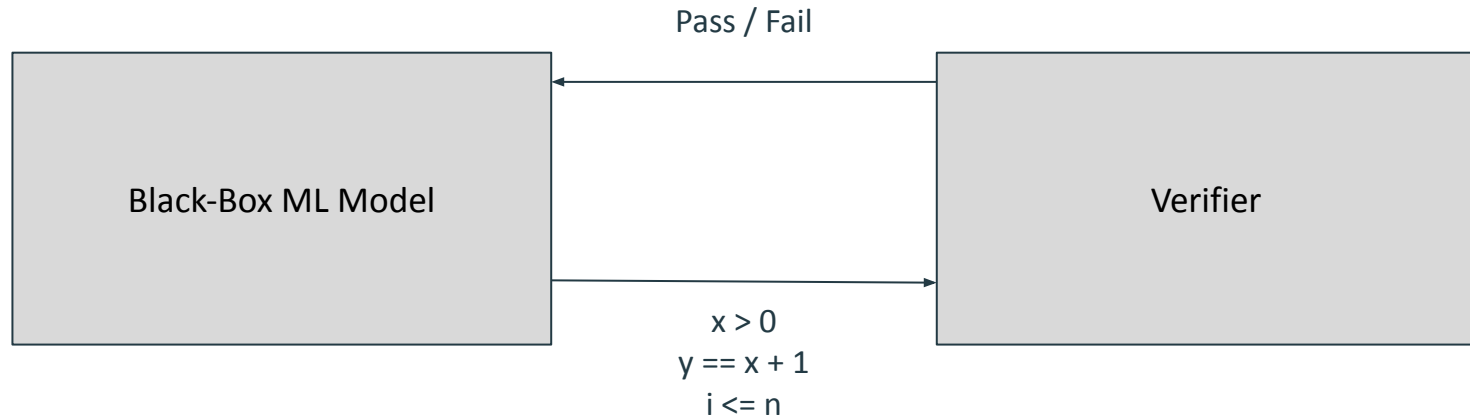
```
method ComputeSum(n: nat) returns (sum: nat)
  ensures sum == n
{
  var i := 0;
  sum := 0;
  while i < n
    invariant i <= n
    invariant sum == i // The Human Bottleneck
  {
    i := i + 1;
    sum := sum + 1;
  }
  // Dafny verifies successfully!
}
```



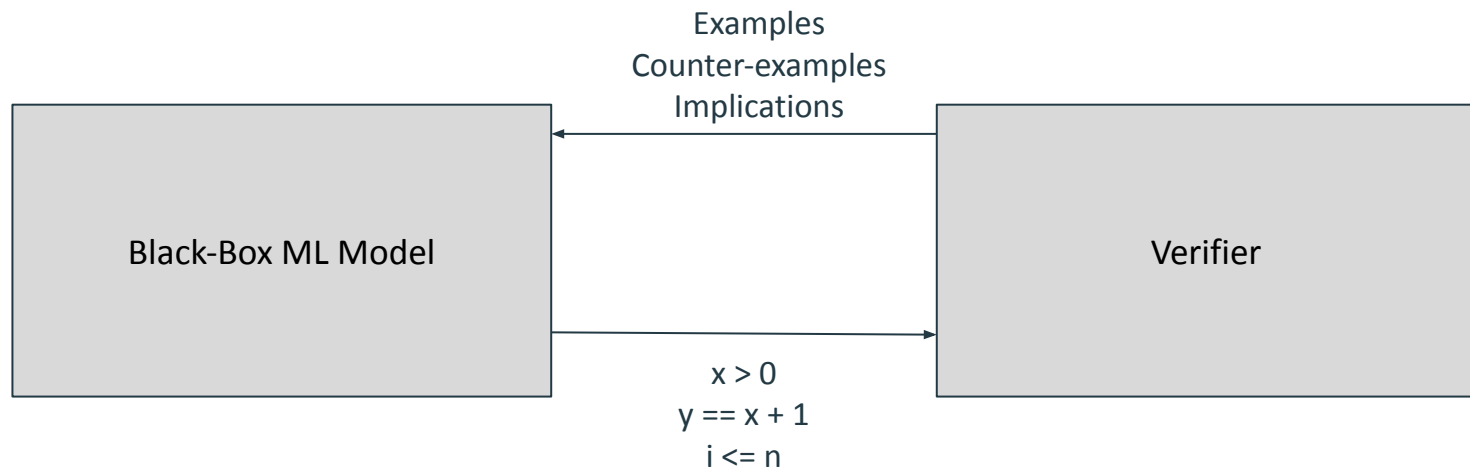
Machine Learning with Invariant Generation



Early machine learning models got stuck guessing invariants because verifiers only provided binary pass/fail feedback.



ICE framework (2014) proves that machine learning can synthesize invariants from verifier counter-examples.



Generative Language Modeling (2020) demonstrated that Transformers can predict formal reasoning steps, but require a checker to filter hallucinations.

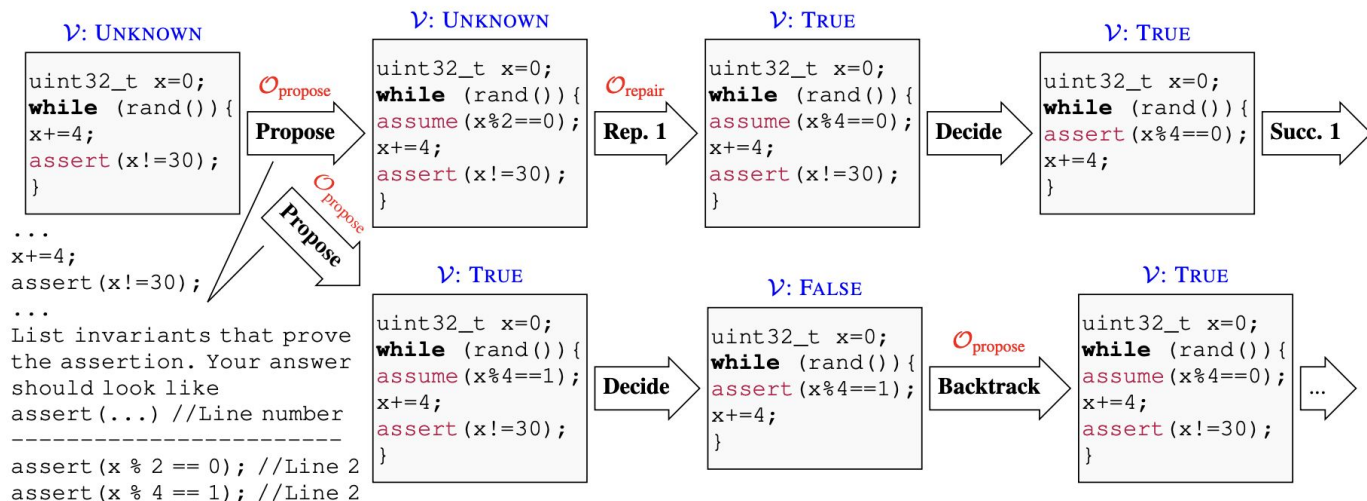


Custom trained transformer on Metamath logic

The model occasionally generated proofs shorter than human experts, but heavily hallucinates without a strict formal checker.

- Fast and easy to implement
- Metamath needs lots of steps compared to other languages
 - LLM would make mistakes after lots of little steps

Lemur (2024) creates a fully automated loop where LLMs propose invariants and automated reasoners strictly validate them.



A hybrid approach combining LLM proposals with SMT solver filtering

Lemur beats traditional verifiers on complex benchmarks, despite LLM constraints.

Configurations	Solved	Time (s)	# proposal
Code2Inv	92	—	-
ESBMC	68	0.34	0
LEMUR(GPT3)	103	35.6	8.6
LEMUR(GPT4)	107	32.9	4.7

(a) The Code2Inv benchmarks.

Configurations	Solved	Time (s)	# proposals
UAUTOMIZER	1	824.3	0
ESBMC	1	675.7	0
LEMUR(GPT3)	14	162.2	8.5
LEMUR(GPT4)	25	234.5	7.2

(b) The 47 SV-COMP benchmarks.

Problems:

- Formatting the output
- Inserting markers on the wrong lines
- Non deterministic behavior would cause the same proof to sometimes succeed and sometimes fail



Interactive Assistance



Lean Copilot (2025) shifts from automated loops to interactive, human-in-the-loop theorem proving assistance.

```
import LeanCopilot
```

```
theorem add_abc (a b c : Nat) : a + b + c = a + c + b := by
```

```
  suggest_tactics
```

Call an LLM to
Generate Tactics

Finish the Proof Directly!

Transform the Goals Errorlessly
But Cannot Finish the Proof

▼ Tactic state

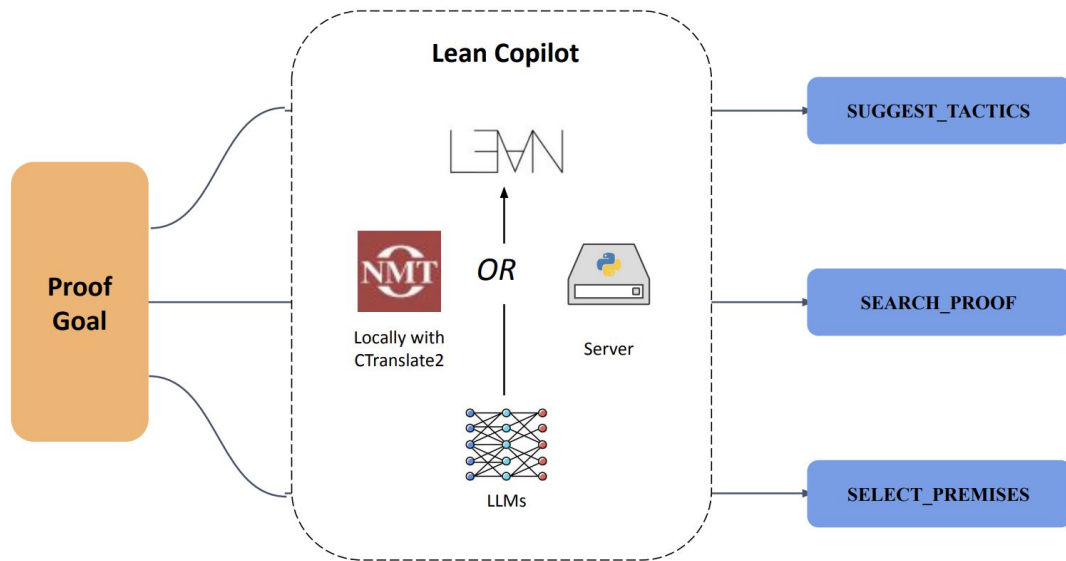
No goals

▼ Suggestions

Try these:

- `apply Nat.add_right_comm`
- `rw [Nat.add_assoc]`
Remaining subgoals:
 $\vdash a + (b + c) = a + c + b$
- `rw [Nat.add_comm]`
Remaining subgoals:
 $\vdash c + (a + b) = a + c + b$
- `simp [Nat.add_assoc]`
Remaining subgoals:
 $\vdash a + (b + c) = a + (c + b)$

Lean Copilot embeds a local language model directly into the proof assistant to suggest mathematically sound tactics.



Integrating LLMs natively into Lean required bypassing standard Python bottlenecks and handling un-tokenizable math syntax.

- Most machine learning is in Python
 - Built custom C++ inference connections
- Lean packs a ton of info in a few small symbols ($\vdash$, λ)
 - Used models without tokenizers
- Analyzes the IDE environment to fetch source code for out-of-scope AI suggestions

By immediately filtering AI hallucinations through Lean's kernel, Copilot significantly improves benchmark proof completion rates.

Method	Avg. # human-entered tactics (↓)	% theorems proved autonomously (↑)	Avg. % proof steps automated (↑)
AESOP	3.86	24.4%	40.1%
SUGGEST_TACTICS	3.10	45.2%	58.3%
SEARCH_PROOFS	2.08	63.7%	74.2%

AI models provide search heuristics, while formal systems guarantee absolute soundness.

AI's Role

- Heuristic Search
- Invariant Generation
- Rapid Proposing

Verifier's Role

- Rigorous Checking
- Absolute Soundness
- Filtering Hallucinations

The current big challenge is getting flexible language models to speak the strict language of mathematical verifiers

- Translating dense SMT solver errors into actionable LLM prompts
- Scaling from simple arrays to complex, heap-allocated data structures
- Overcoming LLM context window limits during iterative proof refinement

Papers

- 1969 | Hoare, C.A.R. - An Axiomatic Basis for Computer Programming. Communications of the ACM
- 2010 | Leino, K.R.M. - Dafny: An Automatic Program Verifier for Functional Correctness. LPAR
- 2014 | Garg, P., et al. - ICE: A Robust Framework for Learning Invariants. Computer Aided Verification (CAV)
- 2020 | Polu, S., & Sutskever, I. - Generative Language Modeling for Automated Theorem Proving. arXiv
- 2024 | Wu, H., et al. - LEMUR: Integrating Large Language Models in Automated Program Verification. ICLR
- 2025 | Song, P., Yang, K., & Anandkumar, A. - Lean Copilot: Large Language Models as Copilots for Theorem Proving in Lean. arXiv / ICLR

Questions?